

04 — Models: architectures, normalisation, evaluation

18 September 2026

1 Theory

Aim

- One network, two feature arms, two tasks
 - **PH arm**: diagrams of 03 $\rightarrow$ persistence images
 - **Classical arm**: summary functions L, F, G, J on a radius grid
 - **Classify**: 5-way family (Poisson, Thomas, nested, Matérn II, LGCP)
 - **Params**: the family’s own parameters, one model per family
- Everything after the feature step is shared $\Rightarrow$ arms comparable by construction
- No regime information enters training; regimes are applied afterwards to the saved test predictions

Inputs

- **Persistence image** (Adams et al. 2017), one per (filtration, homology dim)
 - Each pair $\rightarrow$ Gaussian in (birth, persistence), weight = persistence (linear)
 - Pixel = exact Gaussian mass over its box (erf), not a point sample; mass outside the box dropped
 - 64×64 ; $\sigma = 1$ pixel on both axes (one absolute σ would distort with the axis ratio)
 - 1-D (persistence axis only, 64 bins) where births are all 0: Rips and alpha H_0 . DTM H_0 (births $2f(x)$) and every H_1 are 2-D
- **Summary functions**, 512 samples each
 - $L(r) - r$ (01), $F, G, J = (1 - G)/(1 - F)$
 - **Grid fixed**: $r \in (0, 0.25]$, spatstat’s axis, the only one comparable to VIHRS
 - **Grid sqrt_n_u2**: $r = u/\sqrt{n}$, $u \in (0, 2]$. F, G saturate near the mean NN distance; on **fixed** they sit at 1.0 over 62%/66% of samples, on **sqrt_n** 24%/30%
- **Covariate**: $\log n$ (realised, observable), one scalar

Architecture (PHNet)

- **Encoder** per input key (1-D or 2-D, following the input’s rank):

[conv₃ 32 $\rightarrow$ ReLU $\rightarrow$ avgpool₂ $\rightarrow$ drop] $\rightarrow$ [conv₃ 64 $\rightarrow$ ReLU $\rightarrow$ avgpool₂ $\rightarrow$ drop]
 $\rightarrow$ [conv₃ 128 $\rightarrow$ ReLU] $\rightarrow$ adaptive avgpool(4)

then flatten $\rightarrow$ Linear $\rightarrow$ ReLU, embedding dim 64

- 2-D: $64^2 \rightarrow 32^2 \rightarrow 16^2 \rightarrow 4^2$; 1-D: $64 \rightarrow 32 \rightarrow 16 \rightarrow 4$ (curves: $512 \rightarrow \dots \rightarrow 4$)
- Padding 1, stride 1: convs keep resolution; only the pools shrink it
- **Head**: concat(embeddings, $\log n$) $\rightarrow$ MLP [64, 32], ReLU, dropout 0.1 $\rightarrow$ 5 logits or p targets
- **How inputs combine**

- H_0, H_1 : separate encoders, concatenated. Never stacked as channels: each is calibrated on its own box, so one pixel means different things in each (and under Rips/alpha their ranks differ)
- Multi- k (several DTM k): one *shared-weight* encoder applied to each k in turn, embeddings concatenated
- Curves on one grid: stacked as channels of the first conv (index i is the same r in every channel, so the conv can compare them at that r)
- Curves on different grids: one encoder each, meet only after pooling

Design choices

- **Pool to 4, not global**: a conv stack under a global average is translation-invariant for every weight setting. In a persistence image position *is* the measurement (persistence 0.5 vs 2.0). Same blob at two positions: relative embedding distance 0.0002 (global) vs 0.14 (pool 4)
- **Channel-wise dropout** (Dropout1d/2d, $p = 0.2$): neighbouring pixels are correlated, element-wise dropout regularises little
- **ReLU after the embedding**: otherwise it composes with the head's first Linear into one linear map (a rank bottleneck, nothing more)

Parameter counts (classification head):

Input	encoder(s)	head	total
DTM H_0+H_1 (2-D + 2-D)	$2 \times 223,808$	10,565	458,181
Rips/alpha H_0+H_1 (1-D + 2-D)	$63,872 + 223,808$	10,565	298,245
Rips/alpha H_0 (1-D)	63,872	6,469	70,341
L, F, G, J stacked (1-D, 4 ch)	64,160	6,469	70,629
L, F, G, J separate	$4 \times 63,872$	18,757	274,245

2 Normalisation

No normalisation layers in the network (no BatchNorm/LayerNorm). All normalisation is on the input and target side, and every statistic is fitted on **train rows only**, then applied frozen to val and test.

PH arm, in order

1. **Coordinates** $\times \sqrt{n}$: pairs in units of mean point spacing ($1/\sqrt{n}$ on W). Without it a pattern at $\bar{n} = 800$ fills a third of the axis of one at $\bar{n} = 100 \Rightarrow$ effective resolution would track the $\bar{n}$ regime axis
2. **Box** (`fit_imager`): persistence $[0, 1.05 \cdot q_{99}]$, birth $[q_1, 1.05 \cdot q_{99}]$, quantiles of *per-diagram* extrema over train diagrams $\Rightarrow \approx 99\%$ of train diagrams lie wholly inside; clipping is a property of a pattern, not of the busiest corner. One box per (filtration, dim), shared by every pattern
3. **Mass** / n : image per point (H_0 has $n - 1$ pairs). Without it: $\approx 7\times$ mass range across $\bar{n}$
4. **$\sqrt{\cdot}$ per pixel**: linear H_1 images are $\approx 93\%$ empty with a few bright pixels (standardised range $[-0.2, +50]$, Thomas DTM₁₀). No affine map fixes that; $\sqrt{\cdot}$ is variance-stabilising for mass-like quantities and keeps 0 at 0
5. **z-score per channel**: one (mean, s.d.) per filtration channel, pooled over train patterns *and* pixels. Picture only shifted and rescaled, never reshaped per pixel or per pattern. Needed: H_0 and H_1 pixel scales differ by $\approx 10\times$

Example boxes (classify, train, $\sqrt{n}$ units): DTM₁₀ H_0 pers $[0, 3.85]$, birth $[0.60, 7.36]$; H_1 pers $[0, 2.94]$, birth $[1.24, 7.71]$; Rips H_0 pers $[0, 2.81]$.

Classical arm

- z-score per curve (one mean, s.d. over train patterns and radii), stacked or not
- No $\sqrt{\cdot}$: bounded functions ($F, G \in [0, 1]$), not a sparse heavy-tailed measure
- n -scaling via the *grid* only (`sqrtn`): r axis rescaled, values not

Covariate and targets

- $\log n$, z-scored on train
- Params targets: $\log$ where strictly positive on train (all current targets), then z-score per target. Loss: MSE on the standardised $\log$ targets $\Rightarrow$ each target weighted equally regardless of units
- Predictions inverted ($\cdot$ s.d. + mean, $\exp$) and saved in parameter units
- Targets: Poisson $\bar{n}$; Thomas κ, μ, σ ; nested $\kappa, \mu_1, \mu_2, \sigma_1, \sigma_2$; Matérn R, λ_p ; LGCP $\bar{n}, \sigma^2, s$ ($\bar{n}$ in place of $\mu_{\log}$, the only target that can be negative)

Scope of each fit

- Classify: box, z-score, $\log n$ fitted on train rows of all five families pooled
- Params: fitted on that family's train rows only $\Rightarrow$ different boxes per task
- Split fixed by $\theta \Rightarrow$ every statistic seed-independent; features built once, reused by all 10 seeds

3 Evaluation

Split (`simulation/split.py`)

- By θ index, not pattern: train $\theta < 7000$, val $7000 \leq \theta < 8000$, test $8000 \leq \theta < 10\,000$
- Both replicates of a θ in the same split $\Rightarrow$ no parameter vector seen in train and test
- θ_i drawn from its own stream (02) $\Rightarrow$ indices are i.i.d. prior draws, contiguous blocks are a random split
- Same split for every family, task, arm and seed $\Rightarrow$ all runs paired pattern by pattern
- $\theta \geq 10\,000$ (a larger sweep later) $\rightarrow$ train; the test set never changes

	train	val	test
θ per family	7000	1000	2000
Patterns, classify (5 families)	70 000	10 000	20 000
Patterns, params (1 family)	14 000	2 000	4 000

Training

- AdamW, lr 10^{-3} , weight decay $3 \cdot 10^{-4}$, batch 128
- Loss: cross-entropy (classify), MSE on standardised $\log$ targets (params)
- Up to 200 epochs; early stopping on val loss, patience 30; best-val weights restored
- Val is used *only* for early stopping; no hyperparameter search so far (all defaults above)
- Seed: network init, minibatch order, dropout. Features and split identical across seeds

Test metrics (`scripts/regimes.py`)

- Every test pattern's prediction saved with its `case_id`; joined to the manifest for $(\bar{n}, \tilde{\delta})$ afterwards $\Rightarrow$ bins changeable without retraining
- Classify: accuracy; NLL of the true family. Per family, accuracy = that family's recall

- Params: RMSE in log units, per target and pooled over targets (parameters span orders of magnitude, trained in log)
- Cells: 8 $\log\tilde{\delta}$ bins on $[0.1, 4] \times 3 \log\tilde{n}$ bins on $[100, 800]$, plus all marginals. Edges from the prior, shared by every family and method
- Poisson ($\tilde{\delta} = 0$): in no $\tilde{\delta}$ bin; appears in marginal and $\tilde{n}$ rows only
- $\approx 2000/24 \approx 83$ test θ per family per cell; cells with < 10 θ skipped

Uncertainty

- Seeds averaged per pattern first, then replicates averaged per θ
- 95% interval: bootstrap over **test** θ (1000 resamples), replicates resampled together
- **seed_sd**: s.d. of the per-seed cell estimate, reported alongside (optimisation variance only)
- Comparisons (**--reference**): *paired* difference on the same θ resamples. Much tighter than two independent intervals; the comparison the paper makes

4 Implemented pipeline

Code

src/cloudforger/training/data.py	diagrams/curves $\rightarrow$ inputs, normalisation, targets
src/cloudforger/training/model.py	ImageEncoder, PHNet
src/cloudforger/training/train.py	AdamW loop, early stopping, predict
.../vectorization/persistence_images/image.py	imager, box fit, scaling
src/cloudforger/classical/functions.py, curves.py	L, F, G, J on fixed / sqrtn
src/cloudforger/simulation/split.py	split by θ
scripts/train.py	CLI: one (task, features, variant), list of seeds
scripts/regimes.py	predictions $\rightarrow$ results/regimes.csv
slurm/train.sh	array job over the run matrix

Run matrix

- Tasks: classify + params for each of 5 families = 6
- PH: 6 filtrations (Rips, alpha, DTM_{5,10,15,20}) $\times \{H_0, H_1, H_0+H_1\} = 18$
- Classical: 4 specs
 - **L@fixed, F, G, J** (per-function grids, separate encoders)
 - **L, F, G, J** (all **sqrtn**, stacked)
 - all **fixed** (stacked)
 - **L@fixed** alone (the VIHRS feature set, through this network)
- $6 \times (18 + 4) = 132$ array tasks $\times$ 10 seeds = 1320 runs; one GPU each, features built once per task

Output

results/<task>/<group>/<features>/<variant>/seed_<s>/

e.g. classify/all/dtm_k10/h01/seed_1

- **predictions.npz**: case_id, y_true, y_pred (+ posterior; params also standardised values), test patterns only
- **run.json**: arguments, imager boxes, target transform, loss history, git stamp
- **model.pt**: best-val weights
- Resumable: a seed with **run.json** is skipped

Checks (tests/test_training.py)

Check	Result
Image rank follows filtration (Rips/alpha H_0 1-D, else 2-D)	pass
Encoder separates one blob at two positions (untrained, 5 inits)	rel. distance ≈ 0.14 (> 0.02)
$\sqrt{\cdot}$ reduces standardised skew (max/ min) of H_1	pass
Curves on one grid share an encoder; on different grids do not	pass
Params predictions returned in parameter units	pass
Several seeds share one feature build; finished seeds skipped	pass

Status (first wave, 17–18 Sep)

- Classify, PH arm: complete (18 configs $\times$ 10 seeds). Params: Poisson, Thomas, nested, Matérn PH complete; LGCP partial; classical arm only nested/Matérn params so far
- 5-way test accuracy, mean over 10 seeds (seed s.d. 0.001–0.004; chance 0.2):

	Rips	alpha	DTM ₅	DTM ₁₀	DTM ₁₅	DTM ₂₀
H_0	0.494	0.493	0.470	0.477	0.476	0.473
H_1	0.338	0.362	0.318	0.319	0.320	0.319
H_0+H_1	0.489	0.495	0.471	0.472	0.472	0.468

- Pooled over the whole $\tilde{\delta} \in [0.1, 4]$ prior, much of which is below the classical detection floor (02: L-test rejects $\leq 15\%$ at $\tilde{\delta} \leq 0.25$); per-regime tables pending
- Best epoch 7–196; 1-D H_0 runs converge slowest: 5 Matérn params runs (Rips/alpha H_0) hit the 200-epoch cap

5 Notes

Interpretation

- All scale handling is explicit and fitted on train: $\sqrt{n}$ coordinates, $/n$ mass, $\sqrt{\cdot}$, z-score. The network has no internal normalisation to absorb a mis-scaled input
- $\sqrt{n}$ scaling makes the image $\bar{n}$ -invariant in geometry, not in information: $\log n$ is still given, so the head can condition on it
- Box fitted on pooled families for classify: a family whose diagrams reach further than the others’ is clipped more often (up to the 1% coverage)
- Seeds only measure optimisation variance; data variance comes from the θ bootstrap. Seed s.d. (≤ 0.004) is small next to it
- H_1 adds nothing to H_0 for classification in the first wave (-0.005 to $+0.002$)

Scope and limits

- One architecture, one hyperparameter setting, no search; comparisons are between features at fixed capacity (capacity differs across input layouts, see parameter counts)
- Val selects the stopping epoch only; test untouched until evaluation
- Classical `sqrtn` curves: r rescaled, values not; $L(r) - r$ still in absolute units, shrinking $\propto 1/\sqrt{n}$ at fixed u ($\log n$ available to compensate)
- L@fixed here is the VIHRS *feature set* through PHNet, not the paper-faithful VIHRS network
- Multi- k (shared encoder) implemented, not in the current run matrix

Open

- Per-regime tables (`regimes.py`) for the first wave; reference run to compare against
- Whether the 1% pooled-box clipping affects one family disproportionately
- Whether any learned arm beats the classical floor at $\tilde{\delta} \lesssim 0.25$ (cluster families; Matérn on μ_R)
- `regimes.py`: bin Matérn on μ_R instead of $\tilde{\delta}$ (01); computed from the manifest's R , $\bar{n}$, so no retraining
- Capacity-matched comparison (1-D H_0 encoders are $3.5\times$ smaller than 2-D)
- Remaining params runs (LGCP, classical arm)